Practice set: one idea at a time

Decision making, the Legendre Memory Unit, learning, and the SVD (up to Lecture 9 and PN4)

Questions. Solutions are in a separate file. Leave that file closed.

How to use this

Test 4 (Nov 12) covers Practice Notebook 4 and Lectures 7–9, and it uses everything from Tests 1–3 along the way. Part A rebuilds the pieces of the earlier material that PN4 leans on, using the PN4 networks as the examples. Part B goes through the PN4 material one idea at a time: the LMU, PES learning, adaptive control, and the SVD. Part C puts two ideas together in each question. Part D is a 30-minute practice test.

Each section states one idea, then asks two questions. The first question shows the move. The second changes the numbers, the sign, or the setup, so a memorized picture from the first one will not be enough.

1. Read the idea box. Attempt the first question for about ten minutes.
2. Open the solution for that question only. Read the working, close the file, and rewrite the answer on a blank page.
3. Attempt the second question with the solutions closed. If it fails, the gap is the idea, not the arithmetic. Restate the idea box from memory, then try again before you read that solution.
4. When every second question is possible from a blank page, write Part D in 30 minutes, closed book.

Sketch checklist. Before you finish any sketch, check: (1) both axes labelled, with units; (2) every value the question names is marked (thresholds, steady states, crossing times, switch times, noise floors); (3) every event is drawn (input switching, a decision snapping, a target changing, learning switching on); (4) the starting value and the final value are correct, including where saturation at the radius stops a trace. For "why" questions, name the mechanism (which synapse, which decoder, which singular value), not only the symptom.

Conventions. Exponential synapse $h(t) = \tau^{-1}e^{-t/\tau}$, $H(s) = 1/(1 + s\tau)$. A recurrent population with recurrent transform A' on synapse τ and input transform B' implements $\dot{x} = Ax + Bu$ with $A' = \tau A + I$ and $B' = \tau B$. PES uses Nengo's sign: error $\varepsilon =$ actual $-$ target, and the decoder update is $\Delta d_j = -\eta\,\varepsilon\,a_j$. For the SVD, $\mathbf{A}$ is the $N \times n$ activity matrix (one row per sample of x, one column per neuron), as in Lecture 9, so $\gamma = \mathbf{A}^\mathsf{T}\mathbf{A}$ is $n \times n$. (PN4 writes A as neurons $\times$ samples; then the same matrix is AA^T.) Shifted Legendre polynomials on $[0, 1]$: $\tilde{P}_0 = 1$, $\tilde{P}_1 = 2r - 1$, $\tilde{P}_2 = 6r^2 - 6r + 1$, with $\tilde{P}_i(0) = (-1)^i$ and $\tilde{P}_i(1) = 1$. Leave exponentials, logarithms and roots symbolic. Label every sketch axis.

Part A Pieces of Tests 1–3 that PN4 uses

1 The PN4 accumulator is an integrator with a large input gain

Idea for this section

PN4's accumulator has a recurrent identity connection on $\tau = 0.1\,\text{s}$, and an input connection with the default `transform=1` and the default $\tau_{\text{in}} = 5\,\text{ms}$ synapse. Read it backwards with the recipe: $A' = 1$ gives $A = 0$, an integrator, and $B' = 1$ gives $B = B'/\tau = 10\,\text{s}^{-1}$. So the ideal state is $x(t) = \frac{1}{\tau}\int_0^t u$. Because $\tau_{\text{in}} \neq \tau$, the input also leaks straight into the state (PN3): $x(t) \approx \frac{1}{\tau}\int_0^t u + (1 - \frac{\tau_{\text{in}}}{\tau})(h_{\tau_{\text{in}}} * u)(t)$, an offset of about $0.95u$ that appears within a few ms.

1. For constant inputs $u = 0.2, 0.1, -0.1, -0.2$ switched on at $t = 0$, give the ramp slope of $x(t)$, the size of the offset, and the time at which $|x|$ first reaches 0.9, with and without the offset. Sketch all four traces over 0–2 s for an ensemble of radius 1.

2. Someone "fixes" the input using the PN3 recipe and sets the input `transform=tau`. What does the $u = 0.1$ trace look like now, and does it ever reach 0.9 in 2 s? Separately, someone keeps `transform=1` but changes the recurrent synapse to 10 ms. What system is implemented now, and when does $u = 0.1$ reach 0.9? What do the two changes tell you about which parameter sets the speed of accumulation?

2 Why the accumulator stops at the radius

Idea for this section

The recurrent connection does not compute x exactly; it computes the decoded $\hat{f}(x)$. Inside the radius $\hat{f}(x) \approx x$. Beyond it, neurons with positive encoders are near their maximum rates, nothing new is recruited, and the decoders were never fitted there, so $\hat{f}(x)$ bends below x. The real dynamics are $\tau\dot{x} = \hat{f}(x) - x + B'u$, so the state stops where $\hat{f}(x) - x = -B'u$: a stable equilibrium slightly outside the radius.

1. Answer PN4 §1b. What is the mathematical computation the accumulator performs (input to output)? Using a sketch of $\tau\dot{x}$ against x for $u = 0.1$, explain why the value stops changing near $+1$. Why does the $u = 0.2$ trace stop a little higher than the $u = 0.1$ trace?

2. The input is switched off at $t = 1.5\,\text{s}$, after the $u = 0.1$ trace has stopped near 1. What does x do next, ideally and in the neural version? Then suppose the ensemble had radius 2. When would $u = 0.1$ reach 0.9 now, where would it stop, and what is the cost of the larger radius?

3 Noise turns accumulation into a random walk

Idea for this section

Model the accumulator as drift plus noise: $\dot{x} = bu + n(t)$, with n white noise of intensity σ^2. Then $x(t)$ is Gaussian with mean *but* and standard deviation $\sigma\sqrt{t}$. The mean grows like t and the spread like $\sqrt{t}$, so waiting longer makes the sign of x more reliable. This is the drift-diffusion account of decisions. PN4 adds the noise to the neurons' input currents, which also changes the effective tuning curves, so the decoded recurrent function is no longer exactly x.

1. Take $b = 10\,\mathrm{s}^{-1}$ and $\sigma = 0.3\,\mathrm{s}^{-1/2}$. For $u = 0.1$, give the mean and standard deviation of x at $t = 0.5\,\mathrm{s}$, and how many standard deviations the mean is from zero. Repeat for $u = 0.05$. Which input is more likely to be on the wrong side of zero at $t = 0.5$? How long would the $u = 0.05$ case have to accumulate to be as reliable as $u = 0.1$ at $0.5\,\mathrm{s}$?

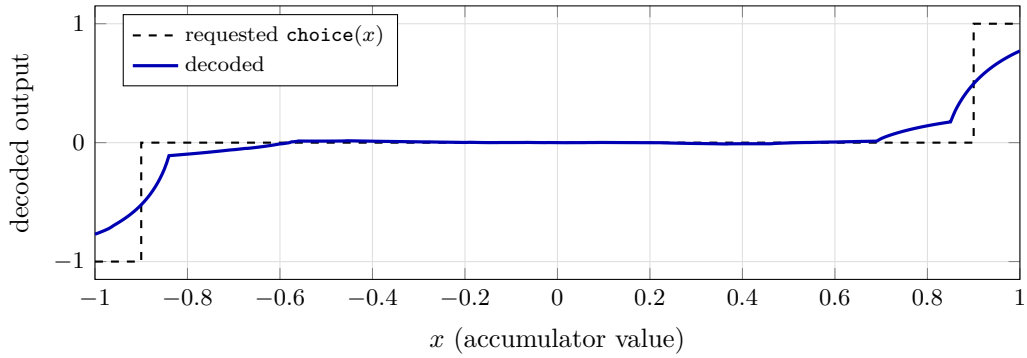
2. In PN4 §1c, with `rms=10` the four traces end at about 0.58, 0.22, −0.33, −0.83 after 2 s instead of near ±1, and they wander. The random-walk model above predicts only extra spread, not smaller values. Explain why strong current noise also makes the accumulator *leak*. (Think about what the decoders were solved for.)

4 Decoding a step function from smooth tuning curves

Idea for this section

A decoded function is a weighted sum of tuning curves, $\hat{f}(x) = \sum_i d_i a_i(x)$, and the tuning curves are smooth, so $\hat{f}$ cannot jump. PN4's `choice` jumps at $x = \pm 0.9$. The decoded version ramps across the jump, misses the corners, and wiggles where it should be flat. A second population can clean this up with its own nonlinearity: if every neuron's intercept is above 0.4, inputs with $|y| < 0.4$ do not make any neuron fire, so the decoded output is exactly zero there.

The plot shows the decoded `choice`(x) from a 50-neuron, radius-1 ensemble (rate approximation).



1. Read off the decoded value at $x = 0.9$ and at $x = 1$, and the largest wiggle for $|x| < 0.8$. Explain each of the three differences from the requested function. The accumulator from Section 1 settles near 1.05; roughly what will the decision population report once it has decided?

2. Answer PN4 §1f: give the decision population `intercepts=Uniform(0.4, 0.9)`. Why is its output now exactly 0 before the decision? What can this population no longer represent, and why doesn't that matter here? Then answer PN4 §1e: if the accumulator and the decision share one 2-D ensemble, what radius does it need, why can't you use the intercept trick, and what changes in the biological claim?

Part B PN4, one idea at a time

5 Functions as vectors: how many numbers does a window need?

Idea for this section

A function can be stored as a vector of numbers: samples, or coefficients on a basis. The Nyquist–Shannon theorem says a signal with no frequencies above B is completely determined, over a window of length T, by $N = 2BT$ equally spaced samples. So the number of state dimensions a memory of the recent past needs is set by its window length times its bandwidth, not by how finely you would like to plot it.

1. How many numbers determine a 5 Hz-limited signal over a 0.5 s window? A 20 kHz-limited audio signal over 1 s? Why are audio sample rates like 44.1 kHz a little above twice the bandwidth?

2. PN4 uses an LMU with $\theta = 0.5\,\text{s}$ to tell a 1 Hz sine from a 2 Hz sine. What is the Nyquist lower bound on q for this input? For white noise limited to 10 Hz with $\theta = 1\,\text{s}$? Why is it impossible to tell 1 Hz from 2 Hz from one instantaneous sample $u(t)$, whatever its value?

6 Basis coefficients when the basis is orthogonal but not orthonormal

Idea for this section

With an orthogonal basis $\{\phi_i\}$ on an interval, the best q-term approximation $\hat{f} = \sum_{i<q} c_i \phi_i$ has $c_i = \langle f, \phi_i \rangle / \langle \phi_i, \phi_i \rangle$. Only for an orthonormal basis is the denominator 1. The shifted Legendre polynomials on $[0, 1]$ are orthogonal with $\langle \tilde{P}_i, \tilde{P}_i \rangle = \int_0^1 \tilde{P}_i^2 = \frac{1}{2i+1}$, so $c_i = (2i+1) \int_0^1 f\, \tilde{P}_i$. Adding terms never changes the earlier coefficients.

1. Check that $\tilde{P}_0$ and $\tilde{P}_1$ are orthogonal on $[0, 1]$ and that $\int_0^1 \tilde{P}_1^2 = \frac{1}{3}$. Find the Legendre coefficients of $f(r) = r$ and confirm that two terms reproduce it exactly.

2. Find c_0, c_1, c_2 for $f(r) = r^2$. Sketch f, its $q = 2$ approximation, and its $q = 3$ approximation on $[0, 1]$, and give the error of the $q = 2$ version at $r = 0$, $\frac{1}{2}$, 1. What coefficients would you get if you forgot the factor $2i + 1$, and what would the "approximation" look like?

7 The LMU is a linear dynamical system

Idea for this section

A pure delay $e^{-s\theta}$ cannot be built from a finite number of synapses, but its Padé approximant of order q can: $\dot{x} = Ax + Bu$ with $x \in \mathbb{R}^q$,

$$\theta A_{ij} = (2i+1) \begin{cases} -1, & i < j \\ (-1)^{i-j+1}, & i \geq j \end{cases}, \qquad \theta B_i = (2i+1)(-1)^i, \qquad i, j = 0, \ldots, q-1.$$

To build it in neurons, use the same recipe as any LTI system: one q-dimensional ensemble, $A' = \tau A + I$ on the recurrent connection and $B' = \tau B$ on the input, both on synapse τ.

1. With $\theta = 0.5\,\text{s}$ and $\tau = 0.1\,\text{s}$, write A, B, A', B' for $q = 1$ and for $q = 2$, with their shapes. Write the two Nengo connections for general q. For $q = 1$, which familiar system is the LMU?

2. For $q = 2$, find the steady state x^* for a constant input u, and say what it means for the stored window. For $q = 1$, give the steady-state amplitude and phase lag of x for the PN4 input (1 Hz sine for 2 s, then 2 Hz), and sketch u and x over 4 s. Explain why a $q = 1$ LMU makes a poor frequency classifier even though the two amplitudes differ.

8 Reading the window back out

Idea for this section

The LMU state holds the Legendre coefficients of the recent window: $u(t-\theta') \approx \sum_{i=0}^{q-1} \tilde{P}_i(\theta'/\theta)\, x_i(t)$ for $0 \le \theta' \le \theta$. Any *linear* function of the window is therefore a fixed linear combination of the x_i: a decoding row vector, used as a `transform`. A delay of θ' uses the row $(\tilde{P}_0(r), \ldots, \tilde{P}_{q-1}(r))$ with $r = \theta'/\theta$. A *nonlinear* function of the window (energy, frequency, category) needs neurons and a decoded `function`.

1. For $q = 6$, write the decoding row for the current input ($\theta' = 0$), for the input θ ago, and for the input $\theta/2$ ago ($\tilde{P}_2(\frac{1}{2}) = -\frac{1}{2}$, $\tilde{P}_4(\frac{1}{2}) = \frac{3}{8}$, and the odd ones are 0 there). Check all three on the steady state for a constant input.

2. Find the decoding row for the window average $\frac{1}{\theta}\int_0^\theta u(t-\theta')\,d\theta'$, and for the difference $u(t) - u(t-\theta)$. Then show that the window energy $\frac{1}{\theta}\int_0^\theta u(t-\theta')^2\,d\theta'$ equals $\sum_i x_i^2/(2i+1)$. Which of the three can be a `transform`, and which needs a decoded `function`?

9 Classifying the window with `eval_points`

Idea for this section

`Connection(a, b, eval_points=x_values, function=target)` solves the decoders by least squares on the states the LMU actually visited, with your labels as targets. It is supervised learning done offline, in one step. The decoder is only trustworthy on states that look like the training states. After any change in the input, the window takes up to θ to fill with the new signal, so states in that interval are a mixture.

1. PN4 §2b: the LMU ($q = 6$, $\theta = 0.5\,\text{s}$, 1000 neurons) sees 1 Hz for 2 s then 2 Hz for 2 s, and the category decoder is trained on that run with targets $+1$ then -1. Sketch the target and a plausible category output over 4 s, marking where you expect it to be worst and why. Why is the output "not perfect" even far from the switch?

2. PN4 §2c: the same decoder is tested on 2 Hz first and 1 Hz second. For about the first 0.25 s it reports the 1 Hz label. Explain why, and give two different fixes. Then predict the shape of the classification RMSE against neuron count (PN4 §2d, 50 to 5000) and against q (PN4 §2e, 1 to 64), and justify each shape.

10 From least squares to an online rule: the delta rule

Idea for this section

For one sample, $E = \frac{1}{2}\big(\sum_i d_i a_i - y^{\mathrm{d}}\big)^2$, so $\partial E/\partial d_j = \varepsilon\, a_j$ with $\varepsilon = \hat{y} - y^{\mathrm{d}}$. Gradient descent gives $d_j \leftarrow d_j - \eta\varepsilon a_j$. After one step on the same sample the new error is $\varepsilon' = \varepsilon\,(1 - \eta\|a\|^2)$. So the step converges if $0 < \eta\|a\|^2 < 2$, is fastest at $\eta\|a\|^2 = 1$, and diverges with growing oscillations beyond 2.

1. Derive Δd_j. Then take two neurons with rates $a = (50, 100)$ Hz, decoders $d = (0.002, 0.004)$, target $y^{\mathrm{d}} = 0.8$, and $\eta = 10^{-5}$. Compute $\hat{y}$, ε, the update to each decoder, the new $\hat{y}$, and the new error. Check that $\varepsilon'/\varepsilon = 1 - \eta\|a\|^2$.

2. Repeat with $\eta = 2 \times 10^{-4}$ and describe the next three values of $\hat{y}$. Give the largest stable η and the one-step-optimal η for this activity vector. Why can't you simply pick the "optimal" η in a real network, where $x(t)$ keeps changing?

11 From decoders to synaptic weights: the PES rule

Idea for this section

Weights are $w_{ij} = \alpha_i \langle e_i, d_j \rangle$. Substituting the delta rule gives the PES rule

$$\Delta w_{ij} = -\eta\, \alpha_i\, \langle e_i, \varepsilon \rangle\, a_j.$$

Everything is local: the post neuron i needs its own gain and encoder, the presynaptic activity a_j, and the error projected onto its encoder, which it receives like an input (a modulatory signal, for example dopamine). In Nengo the error is a separate population, connected to `conn.learning_rule`, and it must compute actual − target.

1. Pre neuron j fires at 80 Hz. Post neuron 1 has $e_1 = +1$, $\alpha_1 = 2$; post neuron 2 has $e_2 = -1$, $\alpha_2 = 1.5$. The error is $\varepsilon = +0.25$ and $\eta = 10^{-4}$. Compute Δw_{1j} and Δw_{2j}. Explain in one sentence each why both changes move the decoded output the right way.

2. PN4 §3b: what happens if the error population computes target − actual instead? Then, for PN4 §3g (learn $x \mapsto (x^2, -x)$), give the dimension of each of `pre`, `post`, `error`, and list every connection with its `function`/`transform`. What should the `initialization` function return?

12 Learning curves: speed, floor, and what you trained on

Idea for this section

Under PES the error falls roughly exponentially and then levels off. The speed scales with η (and with how strongly the needed function is represented, Part C). The floor does not go to zero, because of (i) the finite population's representation error, (ii) spike noise in the activities and the error, and (iii) weight jitter: every update chases the current sample, so a large η keeps overwriting what was learned for other inputs. The rule only learns the inputs it is shown.

1. PN4 §3c: the RMSE over five 2-s chunks of a 10 s run, averaged over 10 runs, is about 0.24, 0.13, 0.083, 0.062, 0.057. Sketch it. Estimate the time constant of the decay, and say what the last value

is close to. Predict what the curve looks like for a 100 s run (PN4 §3d), and for 100 s with $\eta = 10^{-5}$ (PN4 §3e).

2. The training signal is white noise with `rms=0.3`. After 10 s of learning, the input is switched to a slow ramp from -1 to 1. Where along the ramp will the learned identity be worst, and why? Answer PN4 §3f: list four changes that would make the learned function more accurate, and say which of the three error sources each one targets.

13 Adaptive control: learning the force you did not model

Idea for this section

A PD controller $u_{\text{PD}} = k_p(\theta^* - \theta) - k_d\dot{\theta}$ only produces torque when there is an error. If gravity pulls with torque $mgL\cos\theta$, the arm can only be held still where $k_p(\theta^* - \theta) = mgL\cos\theta$, so it always settles short of the target. An adaptive population represents the state $(\theta, \dot{\theta})$ and adds its decoded output u_{ad} to the torque. PES uses the PD signal itself as the error (with a minus sign, because $\varepsilon = \text{actual} - \text{target}$). Learning stops only when the PD controller has nothing left to correct, which means $u_{\text{ad}}(\theta, \dot{\theta}) \approx mgL\cos\theta$.

1. Take $m = L = 1$, $g = 9.8$, $k_p = 25\,\text{s}^{-2}$, $k_d = 10\,\text{s}^{-1}$, $\theta^* = 0$, starting at $\theta = \pi/4$ at rest. Write the equation for the steady angle and estimate it with $\cos\theta \approx 1$. Is the estimate too large or too small in magnitude, and why? Linearize about $\theta = 0$ and say whether the approach is oscillatory. Sketch $\theta(t)$ over 3 s.

2. Now add the adaptive population. Draw the network and give the transform on the connection into `conn.learning_rule`. What should u_{ad} be after learning when $\theta^* = 0$? The target then alternates between 0 and 0.8 rad every 5 s. What is the PD-only steady angle at $\theta^* = 0.8$ (the exact value is 0.446; justify it)? If the adaptive part had learned only a constant torque of 9.8, where would the arm settle at $\theta^* = 0.8$? Why does the population need to represent θ for the controller to work at both targets?

14 What the SVD of γ says, computed by hand

Idea for this section

Every decoder is $\mathbf{D} = \gamma^{-1}\mathbf{A}^{\mathsf{T}}\mathbf{X}$ with the same $\gamma = \mathbf{A}^{\mathsf{T}}\mathbf{A}$, whatever function $\mathbf{X}$ holds. So γ describes everything the population can decode. Write $\gamma = \mathbf{U}\mathbf{S}\mathbf{U}^{\mathsf{T}}$. The columns of $\chi = \mathbf{A}\mathbf{U}$ are new basis functions of x: orthogonal, ordered, with $\|\chi_m\|^2 = S_m$. A function f decodes well if it is built from the χ_m with large S_m. With activity noise of variance σ^2, the coefficient on χ_m is $\Phi_m = \langle \chi_m, f\rangle/(S_m + \sigma^2)$, so directions with $S_m \lesssim \sigma^2$ are shrunk toward zero and are not usable.

1. Two neurons with rates $a_1 = 1 + x$ and $a_2 = 1 - x$ are sampled at $x = -1, 0, 1$. Write $\mathbf{A}$ (3×2) and γ. Find S, $\mathbf{U}$ and $\chi = \mathbf{A}\mathbf{U}$, and describe each χ_m as a function of x. Find the decoder for $f(x) = x$ from $\gamma^{-1}\mathbf{A}^{\mathsf{T}}\mathbf{X}$, and check it. Can this population decode x^2? What is its best attempt?

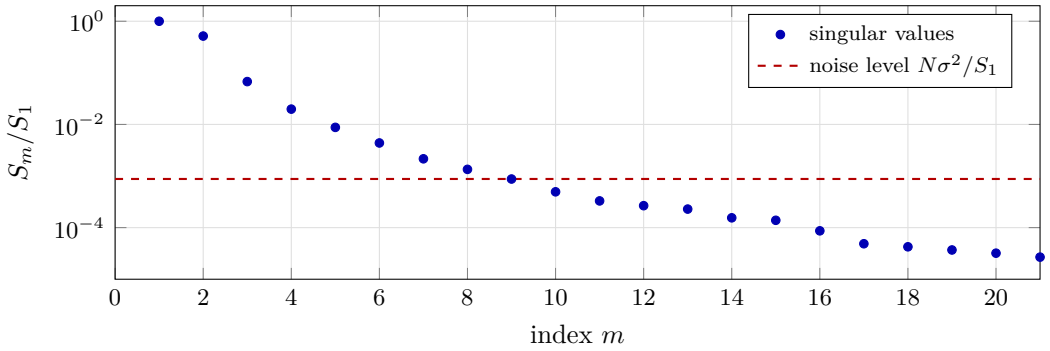
2. Add noise with $\sigma^2 = 4$ (in the same units as S). Use Φ_m to find the decoded $\hat{x}$ and the decoded constant $\hat{1}$ at the three sample points. Which is shrunk more, and why? Now double both rates, $a_1 = 2 + 2x$ and $a_2 = 2 - 2x$, with the same noise. What happens to each singular value and to $\hat{x}$? What does this say about firing rate and noise?

15 The spectrum of a real population

Idea for this section

For a broadly tuned, heterogeneous 1-D LIF population, the $\chi_m(x)$ look like the Legendre polynomials: a near-constant, then linear, quadratic, cubic, and so on, with singular values falling roughly exponentially with order. Low-order functions are well supported and high-order ones are not. In D dimensions, the polynomial terms of order l (including cross terms like x_1x_2) all have singular values of about the same size, and there are $N_S(l, D) = \frac{(l+D-1)!}{l!\,(D-1)!}$ of them.

The plot shows the normalized singular values S_m/S_1 of γ for 100 LIF neurons over $x \in [-1, 1]$ (501 samples), with the noise level $N\sigma^2/S_1$ for $\sigma = 0.1 \max a$, Nengo's default regularization.



1. About how many basis functions are above the noise level? Sketch what you expect $\chi_1, \ldots, \chi_4$ to look like on $[-1, 1]$. Rank x, x^2, $|x|$ and $\text{sign}(x)$ from easiest to hardest to decode, with a reason for each. Connect this to the `choice` plot in Section 4.

2. For a 2-D population, how many singular values do you expect at orders $l = 0, 1, 2, 3, 4$? Sketch the spectrum you expect. How many second-order terms are there in 4-D? Why does decoding a product x_1x_2 from a 2-D population have about the same difficulty as decoding x_1^2? Finally, why does a population whose intercepts are clumped near the ends of the range not follow this tidy ordering?

16 Unsupervised learning: PCA, Hebb, Oja, and STDP

Idea for this section

With no target, a learning rule can still find structure. PCA rotates the data so the first axes carry the most variance: the eigenvectors of the covariance matrix C, ordered by eigenvalue. Hebb's rule $\Delta w_{ij} = \eta a_i a_j$ ("fire together, wire together") always increases weights for positive rates, so it is unstable. Oja's rule $\Delta w_{ij} = \eta(a_i a_j - a_i^2 w_{ij})$ adds a decay that holds $\|w_i\| = 1$, and a linear neuron's weight vector then converges to the first principal component. STDP adds causality: the weight grows if the pre spike comes shortly before the post spike, and shrinks if it comes after.

1. The centred data points are $(2, 1)$, $(-2, -1)$, $(1, 2)$, $(-1, -2)$. Compute $C = \frac{1}{N-1}\sum_k x_k x_k^\mathsf{T}$, its eigenvalues and eigenvectors, and the fraction of variance on the first principal component. Where will Oja's rule take the weight vector of a linear post neuron fed with this data?

2. Show that Hebb's rule with a linear post neuron $a_i = w_i^\mathsf{T} a$ gives $\mathbb{E}[\Delta w_i] = \eta\, C w_i$, and say why this diverges. Show that a fixed point of Oja's rule must satisfy $Cw_i = (w_i^\mathsf{T} C w_i)\, w_i$, and what

that implies. Then, with $W(\Delta t) = \alpha^+ e^{-\Delta t/\tau^+}$ for $\Delta t = t_{\text{post}} - t_{\text{pre}} > 0$ and $-\alpha^- e^{\Delta t/\tau^-}$ otherwise ($\tau^\pm = 10\,\text{ms}$), find the sign and relative size of the change for a pre spike at 10 ms with a post spike at 15 ms, and with a post spike at 5 ms.

Part C Questions that each use two ideas

These are the first questions in the set that are allowed to feel like the test. Attempt all four before opening their solutions.

1. **Why is x^2 learned more slowly than x?** Lecture 8 shows PES learning $f(x) = x^2$ much more slowly than $f(x) = x$ with the same η. Using the delta rule written in the χ basis, show that each coefficient Φ_m approaches its target at a rate proportional to ηS_m. Use the spectrum in Section 15 to estimate how much more slowly the part of x^2 that lives on χ_3 is learned than the linear part.

2. **Accumulate and decide.** Combine Sections 1, 3 and 4. With noise `rms=3`, sketch the accumulator and the decision output on aligned axes for $u = 0.2$ and $u = -0.1$ over 2 s, first with default decision intercepts and then with `Uniform(0.4, 0.9)`. Mark the threshold crossings and the decision times. Does the model decide faster with stronger evidence, and by how much?

3. **An LMU that learns online.** Replace the `eval_points` connection of Section 9 with a PES-learned connection from the LMU ensemble to the category ensemble. Draw the network, say what the error population computes, and name one advantage and one disadvantage compared with `eval_points`.

4. **Three short answers, no equations.** (a) Why are the singular values of γ the same whichever function you plan to decode? (b) Why is a pre-computed decoder not necessarily optimal once the network is spiking, and how can PES do better? (c) In adaptive control, what plays the role of the "target", and why don't we need to know it?

Part D Practice Test 4 (30 minutes, 20 marks, closed book)

Write answers on separate paper. Use symbolic answers and qualitative sketches; label axes and important values. No code is required.

1. **An accumulator** [4 marks]. A 1-D ensemble (radius 1) has a recurrent identity connection on a 50 ms synapse. A constant input u enters through `transform=0.5` on a 5 ms synapse.

   (a) [1] Give A and B of the implemented system $\dot{x} = Ax + Bu$.

   (b) [2] For $u = 0.2$ from $t = 0$, give the time to reach $x = 0.8$ ignoring the input-synapse mismatch, then correct it using the offset $(1 - \tau_{\text{in}}/\tau)B'u$.

   (c) [1] Sketch $x(t)$ for $u = 0.2$ and $u = -0.1$ over 1 s, showing where the traces stop.

2. **The Legendre Memory Unit** [5 marks]. Take $q = 3$, $\theta = 1\,\text{s}$, $\tau = 0.1\,\text{s}$.

   (a) [2] Write A and B, then A' and B'.

(b) [2] Give the decoding rows for $u(t-\theta)$, for $u(t-\theta/2)$, and for the window average. Verify one of them on the steady state for a constant input.

(c) [1] Why can a delay be computed with a `transform`, while "which of two frequencies is in the window" needs a decoded `function`?

3. **PES** [5 marks].

(a) [2] A decoder update uses $a = (30, 40)$ Hz, $d = (0.01, 0.005)$, target 0.1, $\eta = 10^{-4}$. Find ε, Δd, and the new error. Is this η stable for this activity?

(b) [1] Write the PES weight update and say which quantities are local to the post neuron.

(c) [2] Draw a network that learns $f(x) = 1 - x$ online from a stimulus $x(t)$, labelling dimensions and the transform/function on each connection. What goes wrong if the error sign is flipped?

4. **SVD** [4 marks]. Two neurons, $a_1 = 2 + x$ and $a_2 = 2 - x$, are sampled at $x = -1, 0, 1$.

(a) [2] Compute $\gamma = \mathbf{A}^\mathsf{T}\mathbf{A}$, its singular values and vectors, and $\chi = \mathbf{AU}$. Describe each χ_m.

(b) [2] With noise $\sigma^2 = 4$, which of the two basis functions is at the edge of usability? What is the decoded $\hat{x}$ at $x = 1$? State the general rule linking S_m, σ^2 and usability.

5. **Adaptive control** [2 marks]. A PD-controlled arm ($k_p = 25$, $mgL = 9.8$) with target $\theta^* = 0$ settles at -0.37 rad. An adaptive ensemble is added. State the error signal (with sign) driving its learning rule, and the function it ends up computing.

Mark Part D yourself against the solutions, and redo every section from Parts A–C where you lost a mark.